

Punto 3.6 - Agrupaciones con proyección

Práctica

Ejercicio: Añadir agrupaciones con proyección al repositorio de órdenes del proyecto AceriaData. Crear métodos que agrupen por cliente, por estado, por mes y por cliente y estado. Incluir proyecciones con colecciones internas, filtros Having y agregaciones múltiples. Crear un caso de uso que ejecute estas agrupaciones y analice el SQL generado.

Contexto del proyecto: En el punto 3.5 se añadieron consultas de agregación con Count, Sum, Average, Min, Max y GroupBy. En este punto se profundiza en las agrupaciones con proyección, incluyendo agrupaciones por múltiples claves, filtros Having y proyecciones de grupos. Estas agrupaciones se usarán en el Módulo 4 para las optimizaciones de rendimiento.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Crear el DTO ResumenPorClienteConOrdenesDto

Crear el archivo src/AceriaData.Application/Dtos/ResumenPorClienteConOrdenesDto.cs:

```
namespace AceriaData.Application.Dtos;

public class ResumenPorClienteConOrdenesDto
{
    public string Cliente { get; set; } = string.Empty;
    public int TotalOrdenes { get; set; }
    public List<OrdenResumenDto> Ordenes { get; set; } = new();
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class ResumenPorClienteConOrdenesDto → declara el DTO.

Línea 5: public string Cliente { get; set; } = string.Empty; → cliente.

Línea 6: public int TotalOrdenes { get; set; } → total de órdenes.

Línea 7: public List Ordenes { get; set; } = new(); → colección de órdenes.

Error común: si la colección no se inicializa con new(), se produce una NullReferenceException al intentar añadir elementos.

Paso 3: Crear el DTO ResumenPorClienteYEstadoDto

Crear el archivo src/AceriaData.Application/Dtos/ResumenPorClienteYEstadoDto.cs:

```
namespace AceriaData.Application.Dtos;

public class ResumenPorClienteYEstadoDto
{
    public string Cliente { get; set; } = string.Empty;
    public string Estado { get; set; } = string.Empty;
    public int TotalOrdenes { get; set; }
    public decimal PesoTotal { get; set; }
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class ResumenPorClienteYEstadoDto → declara el DTO.

Línea 5: public string Cliente { get; set; } = string.Empty; → cliente.

Línea 6: public string Estado { get; set; } = string.Empty; → estado.

Línea 7: public int TotalOrdenes { get; set; } → total de órdenes.

Línea 8: public decimal PesoTotal { get; set; } → peso total.

Error común: si el DTO usa int para el peso total y el valor real es decimal, se produce una pérdida de precisión. Se debe usar decimal para los pesos.

| Paso 4: Añadir los métodos de agrupación a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    OrdenFabricacion? ObtenerPorId(int id);
    List<OrdenFabricacion> ObtenerTodas();
    IQueryable<OrdenFabricacion> ObtenerQueryable();
    OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden);
    List<OrdenFabricacion> ObtenerPendientesPorCliente(string cliente);
    List<OrdenFabricacion> ObtenerPorEstadoOrdenadasPorFecha(string estado);
    List<OrdenFabricacion> ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta);
    List<OrdenFabricacion> ObtenerPorClienteOrdenadas(string cliente);
    List<OrdenFabricacion> ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta);

    List<string> ObtenerClientesUnicos();
    List<OrdenResumenDto> ObtenerResumenes();
    List<OrdenResumenDto> ObtenerResumenesPorEstado(string estado);
    List<OrdenConTotalesDto> ObtenerOrdenesConTotales();
    List<OrdenConPlanchasDto> ObtenerOrdenesConPlanchas();
    List<OrdenConDetalleDto> ObtenerOrdenesConDetalle();
    List<OrdenConTotalesDto> ObtenerOrdenesConTotalesDetallados();
    List<OrdenCompletaDto> ObtenerOrdenesCompletas();

    int ContarOrdenes();
    int ContarOrdenesPorEstado(string estado);
    bool ExisteAlgunaOrden();
    bool TodasLasOrdenesPendientes();
    decimal ObtenerPesoTotalDePlanchas();
    double ObtenerPesoPromedioDePlanchas();
    decimal ObtenerPesoMinimoDePlanchas();
    decimal ObtenerPesoMaximoDePlanchas();
    DateTime ObtenerFechaMasAntigua();
    DateTime ObtenerFechaMasReciente();

    List<ResumenPorClienteDto> ObtenerResumenPorCliente();
    List<ResumenPorEstadoDto> ObtenerResumenPorEstado();
    List<ResumenPorClienteDto> ObtenerClientesConMasDeUnaOrden();
    List<ResumenMensualDto> ObtenerResumenMensual();

    List<ResumenPorClienteConOrdenesDto> ObtenerResumenPorClienteConOrdenes();
    List<ResumenPorClienteYEstadoDto> ObtenerResumenPorClienteYEstado();
    List<ResumenPorClienteYEstadoDto> ObtenerResumenPorClienteYEstadoConFiltro();
    List<ResumenPorClienteDto> ObtenerResumenPorClienteConMultiplesAgregaciones();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: `using AceriaData.Application.Dtos;` → importa los DTOs.

Línea 2: `using AceriaData.Domain.Entities;` → importa las entidades.

Línea 4: `namespace AceriaData.Application.Interfaces;` → declara el espacio de nombres.

Línea 6: `public interface IOrdenRepositorio` → declara la interfaz.

Línea 8: `OrdenFabricacion? ObtenerPorId(int id);` → método que obtiene por Id.

Línea 9: `List ObtenerTodas();` → método que obtiene todas.

Línea 10: `IQueryable ObtenerQueryable();` → método que devuelve IQueryable.

Línea 11: `OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden);` → método que obtiene por número.

Línea 12: `List ObtenerPendientesPorCliente(string cliente);` → método que obtiene las pendientes de un cliente.

Línea 13: `List ObtenerPorEstadoOrdenadasPorFecha(string estado);` → método que obtiene por estado ordenadas.

Línea 14: `List ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta);` → método que obtiene por rango de fechas.

Línea 15: `List ObtenerPorClienteOrdenadas(string cliente);` → método que obtiene por cliente ordenadas.

Línea 16: `List ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta);` → método que combina cliente y rango.

Línea 18: `List ObtenerClientesUnicos();` → método que obtiene los clientes únicos.

Línea 19: `List ObtenerResumenes();` → método que obtiene los resúmenes.

Línea 20: `List ObtenerResumenesPorEstado(string estado);` → método que obtiene los resúmenes por estado.

Línea 21: `List ObtenerOrdenesConTotales();` → método que obtiene las órdenes con totales.

Línea 22: `List ObtenerOrdenesConPlanchas();` → método que obtiene las órdenes con planchas.

Línea 23: `List ObtenerOrdenesConDetalle();` → método que obtiene las órdenes con detalle.

Línea 24: `List ObtenerOrdenesConTotalesDetallados();` → método que obtiene las órdenes con totales detallados.

Línea 25: `List ObtenerOrdenesCompletas();` → método que obtiene las órdenes completas.

Línea 27: `int ContarOrdenes();` → método que cuenta las órdenes.

Línea 28: `int ContarOrdenesPorEstado(string estado);` → método que cuenta las órdenes por estado.

Línea 29: `bool ExisteAlgunaOrden();` → método que comprueba si existe alguna orden.

Línea 30: `bool TodasLasOrdenesPendientes();` → método que comprueba si todas las órdenes están pendientes.

Línea 31: `decimal ObtenerPesoTotalDePlanchas();` → método que obtiene el peso total.

Línea 32: `double ObtenerPesoPromedioDePlanchas();` → método que obtiene el peso promedio.

Línea 33: `decimal ObtenerPesoMinimoDePlanchas();` → método que obtiene el peso mínimo.

Línea 34: `decimal ObtenerPesoMaximoDePlanchas();` → método que obtiene el peso máximo.

Línea 35: `DateTime ObtenerFechaMasAntigua();` → método que obtiene la fecha más antigua.

Línea 36: `DateTime ObtenerFechaMasReciente();` → método que obtiene la fecha más reciente.

Línea 38: `List ObtenerResumenPorCliente();` → método que obtiene el resumen por cliente.

Línea 39: `List ObtenerResumenPorEstado();` → método que obtiene el resumen por estado.

Línea 40: `List ObtenerClientesConMasDeUnaOrden();` → método que obtiene los clientes con más de una orden.

Línea 41: `List ObtenerResumenMensual();` → método que obtiene el resumen mensual.

Línea 43: `List ObtenerResumenPorClienteConOrdenes();` → método que obtiene el resumen por cliente con órdenes.

Línea 44: `List ObtenerResumenPorClienteYEstado();` → método que obtiene el resumen por cliente y estado.

Línea 45: `List ObtenerResumenPorClienteYEstadoConFiltro();` → método que obtiene el resumen por cliente y estado con filtro Having.

Línea 46: `List ObtenerResumenPorClienteConMultiplesAgregaciones();` → método que obtiene el resumen por cliente con múltiples agregaciones.

Línea 48: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.

Línea 49: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o DTOs concretos.

■ Paso 5: Implementar los métodos de agrupación en el repositorio

Modificar `src/AceriaData.Infrastructure.Repositories/OrdenRepositorio.cs`:

```

public List<ResumenPorClienteConOrdenesDto> ObtenerResumenPorClienteConOrdenes()
{
    return _context.OrdenesFabricacion
        .GroupBy(o => o.Cliente)
        .Select(g => new ResumenPorClienteConOrdenesDto
        {
            Cliente = g.Key,
            TotalOrdenes = g.Count(),
            Ordenes = g.Select(o => new OrdenResumenDto
            {
                NumeroOrden = o.NumeroOrden,
                Cliente = o.Cliente,
                Estado = o.Estado,
                FechaCreacion = o.FechaCreacion
            }).ToList()
        })
        .OrderBy(r => r.Cliente)
        .ToList();
}

public List<ResumenPorClienteYEstadoDto> ObtenerResumenPorClienteYEstado()
{
    return _context.OrdenesFabricacion
        .GroupBy(o => new { o.Cliente, o.Estado })
        .Select(g => new ResumenPorClienteYEstadoDto
        {
            Cliente = g.Key.Cliente,
            Estado = g.Key.Estado,
            TotalOrdenes = g.Count(),
            PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso))
        })
        .OrderBy(r => r.Cliente)
        .ThenBy(r => r.Estado)
        .ToList();
}

public List<ResumenPorClienteYEstadoDto> ObtenerResumenPorClienteYEstadoConFiltro()
{
    return _context.OrdenesFabricacion
        .GroupBy(o => new { o.Cliente, o.Estado })
        .Where(g => g.Count() > 1)
        .Select(g => new ResumenPorClienteYEstadoDto
        {
            Cliente = g.Key.Cliente,
            Estado = g.Key.Estado,
            TotalOrdenes = g.Count(),
            PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso))
        })
        .OrderBy(r => r.Cliente)
        .ThenBy(r => r.Estado)
        .ToList();
}

public List<ResumenPorClienteDto> ObtenerResumenPorClienteConMultiplesAgregaciones()
{
    return _context.OrdenesFabricacion
        .GroupBy(o => o.Cliente)
        .Select(g => new ResumenPorClienteDto
        {
            Cliente = g.Key,
            TotalOrdenes = g.Count(),
            TotalPlanchas = g.Sum(o => o.Planchas.Count()),
            PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso)),
            FechaMasReciente = g.Max(o => o.FechaCreacion)
        })
        .OrderBy(r => r.Cliente)
        .ToList();
}

```

Línea 1: public List ObtenerResumenPorClienteConOrdenes() → declara el método que obtiene el resumen por cliente con órdenes.

Línea 3: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 4: `.GroupBy(o => o.Cliente)` → agrupa por cliente.

Línea 5: `.Select(g => new ResumenPorClienteConOrdenesDto` → proyecta cada grupo al DTO.

Línea 6: `Cliente = g.Key,` → asigna el cliente.

Línea 7: `TotalOrdenes = g.Count(),` → cuenta las órdenes del grupo.

Línea 8: `Ordenes = g.Select(o => new OrdenResumenDto` → proyecta la colección interna de órdenes.

Línea 9: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 10: `Cliente = o.Cliente,` → asigna el cliente.

Línea 11: `Estado = o.Estado,` → asigna el estado.

Línea 12: `FechaCreacion = o.FechaCreacion` → asigna la fecha.

Línea 13: `}).ToList()` → materializa la colección interna.

Línea 14: `}` → cierra la proyección.

Línea 15: `.OrderBy(r => r.Cliente)` → ordena por cliente.

Línea 16: `.ToList();` → materializa la consulta.

Línea 19: `public List ObtenerResumenPorClienteYEstado()` → declara el método que agrupa por cliente y estado.

Línea 21: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 22: `.GroupBy(o => new { o.Cliente, o.Estado })` → agrupa por cliente y estado.

Línea 23: `.Select(g => new ResumenPorClienteYEstadoDto` → proyecta cada grupo al DTO.

Línea 24: `Cliente = g.Key.Cliente,` → asigna el cliente.

Línea 25: `Estado = g.Key.Estado,` → asigna el estado.

Línea 26: `TotalOrdenes = g.Count(),` → cuenta las órdenes del grupo.

Línea 27: `PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso))` → suma el peso de las planchas del grupo.

Línea 28: `}` → cierra la proyección.

Línea 29: `.OrderBy(r => r.Cliente)` → ordena por cliente.

Línea 30: `.ThenBy(r => r.Estado)` → ordena por estado.

Línea 31: `.ToList();` → materializa la consulta.

Línea 34: `public List ObtenerResumenPorClienteYEstadoConFiltro()` → declara el método que agrupa por cliente y estado con filtro Having.

Línea 36: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 37: `.GroupBy(o => new { o.Cliente, o.Estado })` → agrupa por cliente y estado.

Línea 38: `.Where(g => g.Count() > 1)` → filtra los grupos con más de una orden. Se traduce a HAVING.

Línea 39: `.Select(g => new ResumenPorClienteYEstadoDto` → proyecta cada grupo al DTO.

Línea 40: `Cliente = g.Key.Cliente,` → asigna el cliente.

Línea 41: `Estado = g.Key.Estado,` → asigna el estado.

Línea 42: `TotalOrdenes = g.Count(),` → cuenta las órdenes del grupo.

Línea 43: `PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso))` → suma el peso de las planchas del grupo.

Línea 44: `}` → cierra la proyección.

Línea 45: `.OrderBy(r => r.Cliente)` → ordena por cliente.

Línea 46: `.ThenBy(r => r.Estado)` → ordena por estado.

Línea 47: `.ToList();` → materializa la consulta.

Línea 50: `public List ObtenerResumenPorClienteConMultiplesAgregaciones()` → declara el método que agrupa por cliente con múltiples agregaciones.

Línea 52: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 53: `.GroupBy(o => o.Cliente)` → agrupa por cliente.

Línea 54: `.Select(g => new ResumenPorClienteDto` → proyecta cada grupo al DTO.

Línea 55: `Cliente = g.Key,` → asigna el cliente.

Línea 56: `TotalOrdenes = g.Count(),` → cuenta las órdenes del grupo.

Línea 57: `TotalPlanchas = g.Sum(o => o.Planchas.Count()),` → suma las planchas del grupo.

Línea 58: `PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso)),` → suma el peso de las planchas del grupo.

Línea 59: `FechaMasReciente = g.Max(o => o.FechaCreacion)` → obtiene la fecha más reciente del grupo.

Línea 60: `}` → cierra la proyección.

Línea 61: `.OrderBy(r => r.Cliente)` → ordena por cliente.

Línea 62: `.ToList();` → materializa la consulta.

Error común: si se proyecta una colección interna dentro de un `GroupBy`, EF Core puede ejecutar una consulta adicional por cada grupo. Se debe revisar el SQL generado y considerar alternativas como `SelectMany` o cargar los datos en una sola consulta.

| Paso 6: Crear el caso de uso de agrupaciones con proyección

Crear el archivo `src/AceriaData.Application/UseCases/AgrupacionesUseCase.cs`:

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class AgrupacionesUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public AgrupacionesUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== AGRUPACIONES CON PROYECCIÓN ===");

        DemostrarResumenPorClienteConOrdenes();
        DemostrarResumenPorClienteYEstado();
        DemostrarResumenPorClienteYEstadoConFiltro();
        DemostrarResumenConMultiplesAgregaciones();
        DemostrarSqlAgrupacionMultiple();
    }

    private void DemostrarResumenPorClienteConOrdenes()
```

```

{
    Console.WriteLine("\n--- Resumen por cliente con órdenes ---");

    var resmenes = _unidad.Ordenes.ObtenerResumenPorClienteConOrdenes();
    foreach (var resumen in resmenes)
    {
        Console.WriteLine($" {resumen.Cliente} | Total: {resumen.TotalOrdenes}");
        foreach (var orden in resumen.Ordenes)
        {
            Console.WriteLine($" {orden.NumeroOrden} | {orden.Estado} | {orden.FechaCreacion:dd/
MM/yyyy}");
        }
    }
}

private void DemostrarResumenPorClienteYEstado()
{
    Console.WriteLine("\n--- Resumen por cliente y estado ---");

    var resmenes = _unidad.Ordenes.ObtenerResumenPorClienteYEstado();
    foreach (var resumen in resmenes)
    {
        Console.WriteLine($" {resumen.Cliente} | {resumen.Estado} | Órdenes: {resumen.TotalOrdenes} | Peso:
{resumen.PesoTotal} kg");
    }
}

private void DemostrarResumenPorClienteYEstadoConFiltro()
{
    Console.WriteLine("\n--- Resumen por cliente y estado (con más de una orden) ---");

    var resmenes = _unidad.Ordenes.ObtenerResumenPorClienteYEstadoConFiltro();
    foreach (var resumen in resmenes)
    {
        Console.WriteLine($" {resumen.Cliente} | {resumen.Estado} | Órdenes: {resumen.TotalOrdenes} | Peso:
{resumen.PesoTotal} kg");
    }
}

private void DemostrarResumenConMultiplesAgregaciones()
{
    Console.WriteLine("\n--- Resumen por cliente con múltiples agregaciones ---");

    var resmenes = _unidad.Ordenes.ObtenerResumenPorClienteConMultiplesAgregaciones();
    foreach (var resumen in resmenes)
    {
        Console.WriteLine($" {resumen.Cliente} | Órdenes: {resumen.TotalOrdenes} | Planchas:
{resumen.TotalPlanchas} | Peso: {resumen.PesoTotal} kg | Última: {resumen.FechaMasReciente:dd/MM/
yyyy}");
    }
}

private void DemostrarSqlAgrupacionMultiple()
{
    Console.WriteLine("\n--- SQL generado por una agrupación por múltiples claves ---");

    var consulta = _unidad.Ordenes.ObtenerQueryable()
        .GroupBy(o => new { o.Cliente, o.Estado })
        .Select(g => new
        {
            Cliente = g.Key.Cliente,
            Estado = g.Key.Estado,
            Total = g.Count(),
            PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso))
        });

    var sql = consulta.ToQueryString();
    Console.WriteLine(sql);
}
}

```

Línea 1: `using AceriaData.Application.Interfaces;` → importa las interfaces de la aplicación.

Línea 3: `namespace AceriaData.Application.UseCases;` → declara el espacio de nombres.

Línea 5: `public class AgrupacionesUseCase` → declara el caso de uso.

Línea 7: `private readonly IUnidadDeTrabajo _unidad;` → campo de la unidad de trabajo.

Línea 9: `public AgrupacionesUseCase(IUnidadDeTrabajo unidad)` → constructor.

Línea 11: `_unidad = unidad;` → asigna el parámetro al campo.

Línea 14: `public void Ejecutar()` → declara el método principal.

Línea 16: `Console.WriteLine("=== AGRUPACIONES CON PROYECCIÓN ===");` → muestra la cabecera.

Línea 18: `DemostrarResumenPorClienteConOrdenes();` → llama al método de resumen por cliente con órdenes.

Línea 19: `DemostrarResumenPorClienteYEstado();` → llama al método de resumen por cliente y estado.

Línea 20: `DemostrarResumenPorClienteYEstadoConFiltro();` → llama al método de resumen con filtro Having.

Línea 21: `DemostrarResumenConMultiplesAgregaciones();` → llama al método de resumen con múltiples agregaciones.

Línea 22: `DemostrarSqlAgrupacionMultiple();` → llama al método de SQL de agrupación múltiple.

Línea 25: `private void DemostrarResumenPorClienteConOrdenes()` → declara el método.

Línea 27: `Console.WriteLine("\n--- Resumen por cliente con órdenes ---");` → muestra la cabecera.

Línea 29: `var resúmenes = _unidad.Ordenes.ObtenerResumenPorClienteConOrdenes();` → llama al método del repositorio.

Línea 30: `foreach (var resumen in resúmenes)` → itera sobre los resúmenes.

Línea 32: `Console.WriteLine($" {resumen.Cliente} | Total: {resumen.TotalOrdenes}");` → muestra el cliente y el total.

Línea 33: `foreach (var orden in resumen.Ordenes)` → itera sobre las órdenes.

Línea 35: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Estado} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Línea 39: `private void DemostrarResumenPorClienteYEstado()` → declara el método.

Línea 41: `Console.WriteLine("\n--- Resumen por cliente y estado ---");` → muestra la cabecera.

Línea 43: `var resúmenes = _unidad.Ordenes.ObtenerResumenPorClienteYEstado();` → llama al método del repositorio.

Línea 44: `foreach (var resumen in resúmenes)` → itera sobre los resúmenes.

Línea 46: `Console.WriteLine($" {resumen.Cliente} | {resumen.Estado} | Órdenes: {resumen.TotalOrdenes} | Peso: {resumen.PesoTotal} kg");` → muestra los datos.

Línea 50: `private void DemostrarResumenPorClienteYEstadoConFiltro()` → declara el método.

Línea 52: `Console.WriteLine("\n--- Resumen por cliente y estado (con más de una orden) ---");` → muestra la cabecera.

Línea 54: `var resúmenes = _unidad.Ordenes.ObtenerResumenPorClienteYEstadoConFiltro();` → llama al método del repositorio.

Línea 55: `foreach (var resumen in resúmenes)` → itera sobre los resúmenes.

Línea 57: `Console.WriteLine($" {resumen.Cliente} | {resumen.Estado} | Órdenes: {resumen.TotalOrdenes} | Peso: {resumen.PesoTotal} kg");` → muestra los datos.

Línea 61: `private void DemostrarResumenConMultiplesAgregaciones()` → declara el método.

Línea 63: `Console.WriteLine("\n--- Resumen por cliente con múltiples agregaciones ---");` → muestra la cabecera.

Línea 65: `var resúmenes = _unidad.Ordenes.ObtenerResumenPorClienteConMultiplesAgregaciones();` → llama al método del repositorio.

Línea 66: `foreach (var resumen in resúmenes)` → itera sobre los resúmenes.

Línea 68: `Console.WriteLine($" {resumen.Cliente} | Órdenes: {resumen.TotalOrdenes} | Planchas: {resumen.TotalPlanchas} | Peso: {resumen.PesoTotal} kg | Última: {resumen.FechaMasReciente:dd/MM/yyyy}");` → muestra los datos.

Línea 72: `private void DemostrarSqlAgrupacionMultiple()` → declara el método.

Línea 74: `Console.WriteLine("\n--- SQL generado por una agrupación por múltiples claves ---");` → muestra la cabecera.

Línea 76: `var consulta = _unidad.Ordenes.ObtenerQueryable()` → inicia la consulta.

Línea 77: `.GroupBy(o => new { o.Cliente, o.Estado })` → agrupa por cliente y estado.

Línea 78: `.Select(g => new { })` → proyecta a un tipo anónimo.

Línea 79: `Cliente = g.Key.Cliente,` → incluye el cliente.

Línea 80: `Estado = g.Key.Estado,` → incluye el estado.

Línea 81: `Total = g.Count(),` → cuenta las órdenes.

Línea 82: `PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso))` → suma el peso.

Línea 83: `});` → cierra la proyección.

Línea 85: `var sql = consulta.ToQueryString();` → obtiene el SQL.

Línea 86: `Console.WriteLine(sql);` → imprime el SQL.

Error común: si el método `ObtenerQueryable` no está implementado en el repositorio, el código no compila. Se debe asegurar que la interfaz y la implementación lo incluyen.

| Paso 7: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<AgrupacionesUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida `Scoped`.

Error común: si se registra con `Singleton`, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 8: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<AgrupacionesUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 9: Insertar datos de prueba con varias órdenes y planchas

Asegurarse de que hay datos en la base de datos. Modificar el método `Main` para insertar órdenes y planchas:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var orden1 = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) };
    var orden2 = new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) };
    var orden3 = new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Norte", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) };
    var orden4 = new OrdenFabricacion { NumeroOrden = "OF-2024-0004", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 4, 5) };

    context.OrdenesFabricacion.AddRange(orden1, orden2, orden3, orden4);
    context.SaveChanges();

    var plancha1 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso = 370.5m, Activa = true };
    var plancha2 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500, Peso = 280.8m, Activa = true };
    var plancha3 = new PlanchaAcero { OrdenId = orden2.Id, Espesor = 8.0, Ancho = 1000, Largo = 2000, Peso = 125.6m, Activa = true };
    var plancha4 = new PlanchaAcero { OrdenId = orden3.Id, Espesor = 15.0, Ancho = 1800, Largo = 3500, Peso = 450.0m, Activa = true };

    context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3, plancha4);
    context.SaveChanges();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var context = scope.ServiceProvider.GetRequiredService();` → resuelve el `DbContext`.

Línea 4: `context.Database.EnsureDeleted();` → elimina la base.

Línea 5: `context.Database.EnsureCreated();` → crea la base.

Línea 7: `var orden1 = new OrdenFabricacion { ... };` → crea la primera orden.

Línea 8: `var orden2 = new OrdenFabricacion { ... };` → crea la segunda orden.

Línea 9: `var orden3 = new OrdenFabricacion { ... };` → crea la tercera orden.

Línea 10: `var orden4 = new OrdenFabricacion { ... };` → crea la cuarta orden.

Línea 12: `context.OrdenesFabricacion.AddRange(orden1, orden2, orden3, orden4);` → registra las órdenes.

Línea 13: `context.SaveChanges();` → inserta las órdenes.

Línea 15: `var plancha1 = new PlanchaAcero { ... };` → crea la primera plancha.

Línea 16: `var plancha2 = new PlanchaAcero { ... };` → crea la segunda plancha.

Línea 17: `var plancha3 = new PlanchaAcero { ... };` → crea la tercera plancha.

Línea 18: `var plancha4 = new PlanchaAcero { ... };` → crea la cuarta plancha.

Línea 20: `context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3, plancha4);` → registra las planchas.

Línea 21: `context.SaveChanges();` → inserta las planchas.

Error común: si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

| Paso 10: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada agrupación. Se observan los resúmenes por cliente con órdenes, por cliente y estado, con filtro `Having`, con múltiples agregaciones y el SQL generado por una agrupación por múltiples claves.

| Paso 11: Analizar la salida

La salida del programa muestra información como la siguiente:

=== AGRUPACIONES CON PROYECCIÓN ===

--- Resumen por cliente con órdenes ---

```

Constructora del Norte | Total: 3
  OF-2024-0001 | Pendiente | 15/01/2024
  OF-2024-0003 | EnProceso | 10/03/2024
  OF-2024-0004 | Pendiente | 05/04/2024
Constructora del Sur | Total: 1
  OF-2024-0002 | Pendiente | 20/02/2024

```

--- Resumen por cliente y estado ---

```

Constructora del Norte | EnProceso | Órdenes: 1 | Peso: 450.0 kg
Constructora del Norte | Pendiente | Órdenes: 2 | Peso: 651.3 kg
Constructora del Sur | Pendiente | Órdenes: 1 | Peso: 125.6 kg

```

--- Resumen por cliente y estado (con más de una orden) ---

```

Constructora del Norte | Pendiente | Órdenes: 2 | Peso: 651.3 kg

```

--- Resumen por cliente con múltiples agregaciones ---

```

Constructora del Norte | Órdenes: 3 | Planchas: 3 | Peso: 1101.3 kg | Última: 05/04/2024
Constructora del Sur | Órdenes: 1 | Planchas: 1 | Peso: 125.6 kg | Última: 20/02/2024

```

--- SQL generado por una agrupación por múltiples claves ---

```

SELECT [o].[Cliente], [o].[Estado], COUNT(*) AS [Total], COALESCE(SUM((
    SELECT COALESCE(SUM([p].[Peso]), 0.0) FROM [PlanchasAcero] AS [p] WHERE [o].[Id] = [p].[OrdenId]
)), 0.0) AS [PesoTotal]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[IsDeleted] = CAST(0 AS bit)
GROUP BY [o].[Cliente], [o].[Estado]

```

La primera sección muestra el resumen por cliente con órdenes. La segunda sección muestra el resumen por cliente y estado. La tercera sección muestra el resumen con filtro Having. La cuarta sección muestra el resumen con múltiples agregaciones. La quinta sección muestra el SQL generado por una agrupación por múltiples claves.

Observaciones: el SQL generado incluye COUNT(*), SUM, GROUP BY con dos columnas y el filtro global de Soft Delete. La colección interna de órdenes no se traduce a SQL: EF Core la materializa después de ejecutar la consulta. Esto demuestra que las colecciones internas dentro de un GroupBy se cargan en memoria.

Paso 12: Diagnosticar un error común

Modificar el método ObtenerResumenPorClienteConOrdenes para proyectar la colección interna sin ToList:

```

csharp
public List<ResumenPorClienteConOrdenesDto> ObtenerResumenPorClienteConOrdenes()
{
    return _context.OrdenesFabricacion
        .GroupBy(o => o.Cliente)
        .Select(g => new ResumenPorClienteConOrdenesDto
        {
            Cliente = g.Key,
            TotalOrdenes = g.Count(),
            Ordenes = g.Select(o => new OrdenResumenDto
            {
                NumeroOrden = o.NumeroOrden,
                Cliente = o.Cliente,
                Estado = o.Estado,
                FechaCreacion = o.FechaCreacion
            })
        })
        .OrderBy(r => r.Cliente)
        .ToList();
}

```

Resultado esperado: el compilador lanza un error porque la propiedad Ordenes es de tipo List y la proyección devuelve IEnumerable. Se debe llamar a ToList para materializar la colección.

Solución: añadir .ToList() dentro de la proyección.

```

Ordenes = g.Select(o => new OrdenResumenDto
{
    NumeroOrden = o.NumeroOrden,
    Cliente = o.Cliente,
    Estado = o.Estado,
    FechaCreacion = o.FechaCreacion
}).ToList()

```

Resultado esperado con la solución: la consulta compila y la colección se materializa correctamente.

Errores comunes del ejercicio

Error	Causa	Solución
Colección interna sin ToList de la proyección	Se olvidó materializar la colección	Añadir .ToList() dentro
Where antes del GroupBy para HAVING	Se aplica el filtro a las filas	Usar Where después del GroupBy
Propiedad no agrupada del grupo o funciones de agregación	Se proyecta una propiedad que no está en el GroupBy	Usar la clave
N+1 en agrupaciones	Se proyecta una colección interna	Revisar el SQL generado y considerar alternativas
SelectMany sin proyección	Se aplana sin proyectar	Añadir una proyección clara
Agrupación anidada	EF Core ejecuta múltiples consultas	Cargar los datos en una sola consulta

Reto resuelto: Agrupación por mes con proyección de órdenes

Reto: Crear un método en el repositorio que agrupe las órdenes por mes y año, y proyecte cada grupo a un DTO con el año, el mes, el total de órdenes y la lista de números de orden. Añadir el DTO, el método a la interfaz, la implementación y una demostración en el caso de uso.

Solución paso a paso:

Paso 1: Crear el DTO ResumenMensualConOrdenesDto:

```

csharp
namespace AceriaData.Application.Dtos;

public class ResumenMensualConOrdenesDto
{
    public int Anio { get; set; }
    public int Mes { get; set; }
    public int TotalOrdenes { get; set; }
    public List<string> NumerosOrden { get; set; } = new();
}

```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class ResumenMensualConOrdenesDto → declara el DTO.

Línea 5: public int Anio { get; set; } → año.

Línea 6: public int Mes { get; set; } → mes.

Línea 7: public int TotalOrdenes { get; set; } → total de órdenes.

Línea 8: public List NumerosOrden { get; set; } = new(); → lista de números de orden.

| Paso 2: Añadir el método a la interfaz IOrdenRepositorio:

```

List<ResumenMensualConOrdenesDto> ObtenerResumenMensualConOrdenes();

```

Línea 1: List ObtenerResumenMensualConOrdenes(); → declara el método.

Paso 3: Implementar el método en OrdenRepositorio:

```
public List<ResumenMensualConOrdenesDto> ObtenerResumenMensualConOrdenes()
{
    return _context.OrdenesFabricacion
        .GroupBy(o => new { o.FechaCreacion.Year, o.FechaCreacion.Month })
        .Select(g => new ResumenMensualConOrdenesDto
        {
            Anio = g.Key.Year,
            Mes = g.Key.Month,
            TotalOrdenes = g.Count(),
            NumerosOrden = g.Select(o => o.NumeroOrden).ToList()
        })
        .OrderBy(r => r.Anio)
        .ThenBy(r => r.Mes)
        .ToList();
}
```

Línea 1: public List ObtenerResumenMensualConOrdenes() → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .GroupBy(o => new { o.FechaCreacion.Year, o.FechaCreacion.Month }) → agrupa por año y mes.

Línea 5: .Select(g => new ResumenMensualConOrdenesDto → proyecta cada grupo al DTO.

Línea 6: Anio = g.Key.Year, → asigna el año.

Línea 7: Mes = g.Key.Month, → asigna el mes.

Línea 8: TotalOrdenes = g.Count(), → cuenta las órdenes del grupo.

Línea 9: NumerosOrden = g.Select(o => o.NumeroOrden).ToList() → proyecta la colección interna de números de orden.

Línea 10: }) → cierra la proyección.

Línea 11: .OrderBy(r => r.Anio) → ordena por año.

Línea 12: .ThenBy(r => r.Mes) → ordena por mes.

Línea 13: .ToList(); → materializa la consulta.

Paso 4: Añadir la demostración en el caso de uso:

```
private void DemostrarResumenMensualConOrdenes()
{
    Console.WriteLine("\n--- Resumen mensual con órdenes ---");

    var resmenes = _unidad.Ordenes.ObtenerResumenMensualConOrdenes();
    foreach (var resumen in resmenes)
    {
        Console.WriteLine($" {resumen.Anio}-{resumen.Mes:D2} | Total: {resumen.TotalOrdenes}");
        foreach (var numero in resumen.NumerosOrden)
        {
            Console.WriteLine($"    {numero}");
        }
    }
}
```

Línea 1: private void DemostrarResumenMensualConOrdenes() → declara el método.

Línea 3: Console.WriteLine("\n--- Resumen mensual con órdenes ---"); → muestra la cabecera.

Línea 5: var resmenes = _unidad.Ordenes.ObtenerResumenMensualConOrdenes(); → llama al método del repositorio.

Línea 6: foreach (var resumen in resmenes) → itera sobre los resúmenes.

Línea 8: `Console.WriteLine($" {resumen.Anio}-{resumen.Mes:D2} | Total: {resumen.TotalOrdenes}");` → muestra el año, mes y total.

Línea 9: `foreach (var numero in resumen.NumerosOrden)` → itera sobre los números.

Línea 11: `Console.WriteLine($" {numero}");` → muestra el número.

Paso 5: Llamar al método desde Ejecutar:

```
DemostrarResumenMensualConOrdenes();
```

Paso 6: Ejecutar `dotnet run` y verificar que las órdenes se muestran agrupadas por mes y año con la lista de números.

Resultado esperado: las órdenes se muestran agrupadas por año y mes, con el total de órdenes y la lista de números de orden.

Analogía final

Las agrupaciones con proyección en una acería son como los informes agrupados que el jefe de planta pide al archivo central. En lugar de una lista plana de órdenes, el jefe pide un informe agrupado por cliente: para cada cliente, cuántas órdenes tiene y cuáles son. El archivo central agrupa las carpetas por cliente, cuenta las órdenes y prepara una lista con los números de cada orden. Proyectar una colección interna dentro de un grupo es como adjuntar la lista de órdenes al informe del cliente. El filtro `Having` es como pedir solo los clientes que tienen más de una orden. La agrupación por múltiples claves es como agrupar por cliente y estado a la vez. `SelectMany` es como aplanar los grupos en una lista plana. Así funcionan las agrupaciones con proyección en EF Core: se agrupan los datos en el servidor, se proyectan los resúmenes y se adjuntan las colecciones internas cuando es necesario. El resultado es un informe estructurado y eficiente.

Resultado esperado

Al final del ejercicio, deberías haber:

Creado los DTOs `ResumenPorClienteConOrdenesDto` y `ResumenPorClienteYEstadoDto`.

Añadido los métodos de agrupación a la interfaz `IOrdenRepositorio`.

Implementado los métodos en `OrdenRepositorio`.

Creado el caso de uso `AgrupacionesUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con varias órdenes y planchas.

Ejecutado las demostraciones de resumen por cliente con órdenes, por cliente y estado, con filtro `Having` y con múltiples agregaciones.

Analizado el SQL generado por una agrupación por múltiples claves.

Diagnosticado el error de no materializar la colección interna.

Creado el DTO `ResumenMensualConOrdenesDto` y su método.

Conexión con el siguiente punto

En este punto se han añadido agrupaciones con proyección al proyecto `AceriaData`, incluyendo agrupaciones por múltiples claves, filtros `Having`, proyecciones de colecciones internas y agrupaciones con múltiples agregaciones. Se ha analizado el SQL generado y se ha comprobado que las colecciones internas se materializan en memoria después de la agrupación. En el siguiente punto se estudiarán los joins y la navegación en consultas, incluyendo `Join`, `GroupJoin` y la navegación por propiedades de navegación.